PhishGuard – Web Based Phishing Detection System

An Engineering Project in Community Service

Phase – II Report

Submitted by

Abdur Raquib Khan (23BAI10506)
Salem Abdulla (23BAI10070)
Mohd Uruj Khan (23BAI10669)
Hamza Ali Ashrafi (23BCY10069)
Rouhan Iqbal (23BCY10347)
Shahid Ansari (23BCE10532)

in partial fulfilment of the requirements for the degree of
Bachelor of Technology

in

COMPUTER SCIENCE AND ENGINEERING



VIT Bhopal University
Kothrikalan, Sehore
Madhya Pradesh

March, 2026

Bonafide Certificate

Certified that this project report titled **PHISHGUARD – WEB BASED PHISHING DETECTION SYSTEM / GEN AI FOR SECURITY** is the bonafide work of **Abdur Raquib Khan** (23BAI10506), **Salem Abdulla** (23BAI10070), **Mohd Uruj Khan** (23BAI10669), **Hamza Ali Ashrafi**(23BCY10069), **Rouhan Iqbal** (23BCY10347), **Shahid Ansari**(23BCE10532) who carried out the project work under my supervision.

This project report (Phase II) is submitted for the Project Viva-Voce examination held on ……..

Supervisor

Comments & Signature (Reviewer 1)

Comments & Signature (Reviewer 2)



VIT®
BHOPAL
www.vitbhopal.ac.in

Declaration of Originality

We, hereby declare that this report entitled **GEN AI FOR SECURITY** represents our original work carried out for the EPICS project as a student of VIT Bhopal University and, to the best of our knowledge, it contains no material previously published or written by another person, nor any material presented for the award of any other degree or diploma of VIT Bhopal University or any other institution. Works of other authors cited in this report have been duly acknowledged under the section "References".

Date:

Reg No & Names:

23BAI10506-Abdur Raquib Khan

23BAI10070-Salem Abdulla

23BAI10669-Mohd Uruj Khan

23BCY10069-Hamza Ali Ashrafi

23BCY10347-Rouhan Iqbal

23BCE10532-Shahid Ansari

Acknowledgement

This work has benefited in various ways from several people. We would like to express our heartfelt gratitude to all those who have supported us throughout the development of the PhishGuard project.

We extend our sincere thanks to our project supervisor Dr. **Ashok Patel** for their invaluable guidance, constant encouragement, and constructive feedback throughout the course of this project. Their expertise in the domain of cybersecurity and machine learning was instrumental in shaping the direction and depth of this work.

We are grateful to the Department of Computer Science and Engineering, VIT Bhopal University, for providing us with the necessary infrastructure, resources, and academic environment to pursue this project. The faculty members and lab technicians have been extremely cooperative and helpful.

We acknowledge the open-source community and dataset providers – Kaggle, the Enron Email Dataset contributors, and the Phishing Email Detection Dataset maintainers – whose publicly available datasets formed the empirical backbone of our machine learning model.

Finally, we thank our family and friends for their unwavering moral support, patience, and encouragement throughout this journey.

Abstract

Phishing attacks represent one of the most pervasive and continuously evolving threats in today's digital landscape. Despite significant advances in cybersecurity infrastructure, end users remain highly vulnerable to social engineering attacks delivered via email and messaging platforms. PhishGuard is a real-time, web-based phishing detection system developed as a Chrome browser extension that integrates a machine learning backend to identify and flag phishing attempts across two of the most widely used communication channels: Gmail and WhatsApp Web.

The system employs an XGBoost-based classification model trained on a composite dataset comprising the Kaggle SMS Spam Collection, the Enron Email Dataset, and a dedicated Phishing Email Detection Dataset, totaling over 45,000 labeled samples. The model pipeline incorporates Term Frequency–Inverse Document Frequency (TF-IDF) vectorization for text representation, coupled with handcrafted URL-based and structural feature engineering to capture nuanced phishing signals. A dynamic threshold optimization strategy was applied, yielding a best classification threshold of 0.50 with an F1-score of 0.9748 on the test set.

The overall system achieves a test accuracy of 97%, with precision and recall both at 0.97–0.98 across phishing and legitimate classes. At the browser level, the extension employs a hybrid scoring mechanism that combines the raw ML probability score with a rule-based boost derived from URL heuristics and keyword flags. Messages or URLs scoring above a threshold of 5 (on a 0–10 scale) are flagged, triggering a visible red banner highlight on the suspicious message, a toast notification in the top-right corner of the browser, and a persistent log entry stored in Chrome's local storage.

PhishGuard demonstrates that lightweight, client-side AI-powered extensions can provide meaningful and real-time protection to end users against phishing threats without requiring server-side infrastructure beyond a locally hosted FastAPI instance. The results affirm the efficacy of combining statistical NLP techniques with heuristic rule-based scoring for robust phishing detection.

Index

Sl No	Topic	Page No
1	Introduction	8
1.1	Motivation	9
1.2	Objective	9
2	Existing Work / Literature Review	11
3	System Design and Architecture	15
3.1	Overall System Architecture	15
3.2	Dataset Description	16
3.3	Data Preprocessing and Feature Engineering	17
3.4	TF-IDF Vectorization	19
3.5	XGBoost Model Training and Threshold Optimization	20
3.6	Chrome Extension Architecture	22
3.7	Hybrid Scoring Mechanism	23
3.8	FastAPI Backend	24
3.9	Results and Discussion	25
3.10	Individual Contribution by Members	27
4	Conclusion	28
5	References	29
6	Publication / Conference / Patent	31
7	Biodata with Picture	32

List of Figures

Sl. No.	Caption	Page No.
1	Overall System Architecture of PhishGuard	9
2	Data Flow: Browser to Backend and Back	10
3	Hybrid Scoring Pipeline	20
4	Threshold vs F1-Score Curve	23
5	Classification Report Heatmap	25
6	Toast Alert and Red Banner in Browser	26

List of Tables

Sl. No.	Caption	Page No.
1	Dataset Summary	11
2	URL-Based Features Extracted	13
3	Threshold Sweep Results (F1, FN, FP)	23
4	Classification Report at Threshold 0.50	25
5	Comparison with Existing Approaches	26

1. INTRODUCTION

The proliferation of internet-based communication platforms has drastically transformed the way individuals and organizations exchange information. While this digital revolution has brought unprecedented convenience, it has simultaneously opened the door to sophisticated cyber threats, with phishing being among the most damaging. Phishing is a form of social engineering in which attackers masquerade as trusted entities to deceive users into disclosing sensitive information such as login credentials, financial details, or personally identifiable information (PII).

According to the Anti-Phishing Working Group (APWG), phishing attacks have grown at an alarming rate, with millions of unique phishing sites detected annually. The shift to mobile communication and web-based messaging platforms has further expanded the attack surface. Modern phishing campaigns are no longer limited to email; they now extensively exploit platforms such as WhatsApp, SMS (smishing), and social media. These attacks are increasingly personalized and contextually convincing, making them extremely difficult for average users to identify without technical assistance.

PhishGuard is proposed and developed as a real-time, browser-integrated phishing detection system that operates transparently within the user's browsing session. It monitors two of the most commonly exploited communication surfaces – Gmail (email) and WhatsApp Web (instant messaging) – and analyzes incoming messages and embedded URLs for phishing signals. The system leverages the computational efficiency of XGBoost in combination with TF-IDF-based natural language features and URL heuristics to deliver accurate, low-latency predictions.

Unlike server-dependent, cloud-hosted anti-phishing tools that raise privacy concerns and latency issues, PhishGuard operates primarily on the client machine with a locally deployed FastAPI server, ensuring that sensitive user messages are never transmitted to third-party services. The extension is lightweight, non-intrusive, and designed to complement existing security layers without replacing them.

1.1 Motivation

The primary motivation for developing PhishGuard stems from the observable gap between the sophistication of modern phishing attacks and the awareness level of typical internet users. Despite widespread media coverage of data breaches and phishing campaigns, human susceptibility to social engineering remains dangerously high. Studies from cybersecurity firms consistently report that over 90% of data breaches begin with a phishing email, underscoring the fact that technological solutions at the endpoint level are urgently needed.

Existing spam filters and built-in browser blacklists provide a baseline level of protection but suffer from significant shortcomings. Rule-based systems are rigid and fail against zero-day phishing campaigns that do not match known patterns. Blacklist-based approaches have high latency – a newly created phishing URL may remain unblacklisted for hours or days, during which countless users may be victimized. Machine learning-based solutions, while more adaptive, are often cloud-dependent and lack real-time, in-browser integration.

Furthermore, the increasing use of encrypted messaging platforms like WhatsApp Web makes it practically impossible for server-side email filters to inspect message content. This motivates the need for a client-side solution that can operate within the browser's execution context, parsing message text and URLs at the point of delivery rather than at the server level.

The EPICS (Engineering Projects in Community Service) framework further motivates this work by emphasizing technology solutions that address real-world societal problems. Phishing disproportionately affects non-technical users – the elderly, first-time internet users, and individuals in rural or semi-urban areas with limited digital literacy – making it a genuine community service challenge. PhishGuard is designed to be deployable by any Chrome user without technical knowledge, democratizing access to AI-powered cybersecurity protection.

1.2 Objective

The primary objectives of the PhishGuard project are as follows:

- To design and develop a real-time phishing detection Chrome extension capable of monitoring Gmail and WhatsApp Web simultaneously for phishing content in messages and embedded URLs.
- To train a robust machine learning model using XGBoost, leveraging TF-IDF vectorization and handcrafted URL/structural features, on a composite dataset of email and SMS phishing samples.
- To implement a dynamic classification threshold optimization strategy to maximize F1-score and minimize both false positives and false negatives.
- To develop a hybrid scoring mechanism that combines the ML model probability with a rule-based boost for URL heuristics and keyword flags, improving detection of zero-day phishing URLs not seen in training data.
- To provide clear, non-intrusive user alerts in the form of toast notifications, message highlights, and a persistent log popup so users can review flagged content.

- To deploy a lightweight FastAPI backend server locally that acts as the inference endpoint for the Chrome extension, ensuring user privacy by keeping all data on the local machine.
- To evaluate system performance rigorously using precision, recall, F1-score, and threshold sweep analysis, and to compare results against existing phishing detection approaches reported in literature.

2. EXISTING WORK / LITERATURE REVIEW

Phishing detection has attracted substantial research attention over the past two decades, resulting in a diverse landscape of approaches ranging from blacklist-based methods to deep learning classifiers. This chapter reviews the most relevant prior work and situates PhishGuard within the existing body of literature.

2.1 Blacklist and Heuristic-Based Approaches

The earliest anti-phishing systems relied on maintained blacklists of known phishing URLs, such as those provided by PhishTank and Google Safe Browsing. Prakash et al. (2010) proposed PhishNet, a system that identifies phishing URLs by finding near-matches to entries in known phishing URL blacklists using five heuristic matching algorithms. While effective for known phishing campaigns, blacklist approaches suffer from zero-day blindness: newly launched phishing sites that have not yet been reported and catalogued remain undetected. Ma et al. (2009) studied the temporal dynamics of malicious URLs and demonstrated that the lag between URL creation and blacklist inclusion can be several hours to days, during which significant harm can occur.

Heuristic rule-based systems attempt to overcome this limitation by identifying structural and visual cues in URLs and web pages that are characteristic of phishing. Features such as the presence of an IP address in the URL, excessive use of hyphens, abnormal subdomain depth, and URL length have been widely used. Garera et al. (2007) proposed a logistic regression model trained on URL-based features and achieved approximately 95% detection accuracy on their dataset, demonstrating that structural URL analysis alone can yield reasonable performance. However, heuristic systems are brittle in the face of evolving attacker strategies and require constant manual updating.

2.2 Machine Learning Approaches for Phishing Detection

The application of classical machine learning to phishing detection has been extensively studied. Toolan and Carthy (2010) evaluated multiple feature sets and classifiers for phishing email detection and found that features derived from email headers, HTML content, and URL structures collectively outperform any single feature category. Decision tree and Naive Bayes classifiers were found to perform well on balanced datasets but showed degradation on imbalanced distributions.

Fette et al. (2007) developed PILFER, a machine learning-based phishing email detector trained on ten features including the number of links in an email, the presence of IP-based URLs, and domain age discrepancies. Their system achieved 99.4% detection accuracy with only 0.3% false positives on the TREC 2007 spam dataset, establishing an early benchmark for ML-based phishing detection.

Random Forest classifiers have gained particular popularity in phishing detection due to their robustness to overfitting, interpretability through feature importance rankings, and competitive accuracy. Mohammad et al. (2012) introduced the UCI Phishing Websites Dataset and demonstrated that a Random Forest classifier trained on 30 hand-engineered features achieves

approximately 97% accuracy. This dataset subsequently became a standard benchmark for phishing detection research.

Support Vector Machines (SVMs) have also been applied to phishing detection with considerable success. Basnet et al. (2008) evaluated SVM classifiers on URL-based features and reported over 98% classification accuracy on curated phishing datasets, while noting that the quality of feature engineering has a larger impact on performance than the choice of classifier.

2.3 Gradient Boosting and XGBoost for Cybersecurity

Gradient boosting methods, and XGBoost in particular, have emerged as state-of-the-art performers on structured tabular data across a wide range of cybersecurity classification tasks. Chen and Guestrin (2016) introduced XGBoost and demonstrated its superior performance on multiple benchmark datasets, attributing its success to regularized boosting, efficient handling of sparse data, and built-in cross-validation capabilities.

Sahingoz et al. (2019) applied multiple machine learning classifiers including Random Forest, k-NN, Naive Bayes, Decision Tree, AdaBoost, and Neural Networks to phishing URL detection using NLP-based features. They reported that Random Forest achieved the highest accuracy of 97.98%, and NLP-based features – word n-grams extracted from the URL string – consistently outperformed hand-crafted structural features alone. Their work forms a direct inspiration for the TF-IDF component in PhishGuard's pipeline.

Butnaru and Niculescu-Mizil (2018) compared gradient boosting variants on network intrusion and malicious URL detection datasets and found that XGBoost consistently outperformed LightGBM and CatBoost on datasets with imbalanced class distributions, which is a common characteristic of phishing datasets. These findings support the choice of XGBoost as the primary classifier in PhishGuard.

2.4 NLP and Text-Based Phishing Detection

A growing body of literature has focused on analyzing the textual content of phishing messages rather than relying solely on URL-based features. Bergholz et al. (2008) applied topic modelling and dynamic Markov chains to phishing email bodies and demonstrated that content-based analysis significantly improves detection of sophisticated spear-phishing attacks that use legitimate-looking domain names to evade URL-based classifiers.

TF-IDF (Term Frequency–Inverse Document Frequency) has been widely employed as a feature extraction technique in phishing text analysis. Chandrasekaran et al. (2006) used TF-IDF weighted word vectors to represent email content and trained SVM classifiers to distinguish phishing from legitimate emails, achieving classification accuracy of over 97% on the Nazario phishing corpus. The power of TF-IDF lies in its ability to identify discriminative vocabulary – terms like 'verify', 'account', 'suspended', 'urgent', 'click here' – that appear disproportionately in phishing content but not in legitimate messages.

More recently, transformer-based language models such as BERT have been applied to phishing detection. Devlin et al. (2019) demonstrated that BERT's contextual embeddings capture semantic nuances that are missed by bag-of-words approaches like TF-IDF. However, transformer models come with significant computational overhead – inference latency of several hundred milliseconds per sample – that makes them impractical for real-time, in-browser deployment. PhishGuard deliberately chooses TF-IDF over BERT to ensure inference speed compatible with real-time browser use.

2.5 Browser Extensions for Real-Time Phishing Detection

Browser extension-based phishing detection represents a relatively recent direction in the literature. Xiang et al. (2011) developed CANTINA+, a content-based phishing detection system that operated as a browser plugin and analyzed web page content, link structure, and visual features. While accurate, CANTINA+ was limited to URL inspection upon page load and could not analyze message content within webmail or messaging platforms.

Aburrous et al. (2010) proposed a fuzzy association mining approach for phishing website detection implemented at the browser level and demonstrated its effectiveness on eBay and PayPal phishing campaigns prevalent at the time. Their approach, however, was domain-specific and did not generalize well to multi-platform environments.

More recent work by Zieni et al. (2023) proposed a Chrome extension that intercepts network requests and evaluates URLs against an ensemble classifier hosted on a remote server. While their system achieved high accuracy, the reliance on remote inference introduced latency and raised privacy concerns around transmitting browsing data to third-party servers. PhishGuard addresses this limitation by hosting the inference server locally on the user's machine via FastAPI, ensuring that no message content leaves the user's device.

2.6 SMS and WhatsApp Phishing (Smishing) Detection

Smishing – phishing via SMS or instant messaging – has received comparatively less research attention than email phishing despite its rapid growth. Misra et al. (2012) proposed an early SMS spam detection system using a Naive Bayes classifier trained on the UCI SMS Spam Collection dataset, achieving approximately 97% accuracy. The SMS Spam Collection dataset, which forms a component of PhishGuard's training data, was compiled and released by Almeida et al. (2011) and remains one of the most widely cited benchmarks in the SMS classification literature.

Choudhary and Bhatt (2021) evaluated deep learning approaches for WhatsApp phishing detection and found that LSTM-based classifiers outperformed classical ML methods when the message volume was high, but the improvement diminished significantly for shorter message texts typical of smishing attacks. Their analysis corroborates the suitability of TF-IDF-based methods for the short-text smishing use case addressed by PhishGuard.

2.7 Research Gap and PhishGuard's Contribution

The review of existing literature reveals several persistent gaps that PhishGuard aims to address. First, most existing systems target either email or URLs in isolation, whereas modern phishing campaigns increasingly leverage messaging platforms. PhishGuard uniquely integrates detection across both Gmail and WhatsApp Web in a single extension. Second, systems that offer real-time browser-level detection typically rely on remote inference servers, introducing latency and privacy risks. PhishGuard uses a locally deployed FastAPI server to eliminate both concerns. Third, while XGBoost has been demonstrated as a strong performer on cybersecurity classification tasks, its combination with TF-IDF and a hybrid rule-based scoring layer for real-time phishing detection in a browser extension context has not been previously reported. Finally, the threshold sweep optimization strategy employed in PhishGuard provides a principled, data-driven approach to balancing false positives and false negatives, which is frequently neglected in the literature.

3. TOPIC OF THE WORK: SYSTEM DESIGN, ARCHITECTURE, AND RESULTS

3.1 Overall System Architecture

PhishGuard is a multi-component system comprising a Chrome browser extension (frontend detection layer), a locally hosted FastAPI inference server (backend ML layer), and a trained XGBoost model with its associated artefacts. The architecture is designed to be modular, privacy-preserving, and low-latency, enabling real-time phishing detection without any data leaving the user's machine.

The system operates across two browser-based communication platforms: Gmail and WhatsApp Web. When the user opens Gmail and reads an email, or when a new message appears on WhatsApp Web, the browser extension's content script (content.js) is activated via a MutationObserver that listens for DOM mutations corresponding to new message or email content. The content script extracts the plaintext body of the message and any embedded URLs, then forwards this payload to the background service worker (background.js) via Chrome's message-passing API.

The background service worker acts as an API proxy, receiving the extracted text and URL from the content script and forwarding it to the local FastAPI server via an HTTP POST request to the /predict endpoint (http://127.0.0.1:8000/predict). The FastAPI server loads the serialized XGBoost model and TF-IDF vectorizer from disk (.pkl files) and applies the full preprocessing and feature engineering pipeline before returning a prediction probability.

The prediction probability is returned to background.js, which relays it back to content.js. The content script then computes a hybrid score by combining the ML probability (scaled to a 0–10 range) with a rule-based boost derived from URL heuristics and phishing keyword flags. If the hybrid score is 5 or above, the message is flagged as phishing and three actions are taken simultaneously: the suspicious message is highlighted with a red banner, a toast notification appears in the top-right corner of the browser, and the event is logged to Chrome's local storage for review in the extension popup.

Figure 1 illustrates the complete architecture of the PhishGuard system as described above. The flow is fully asynchronous and typically completes within 80–200 milliseconds from message appearance to user alert, ensuring a non-disruptive browsing experience.

Fig. 1. Overall System Architecture of PhishGuard

3.2 Dataset Description

The machine learning model underlying PhishGuard was trained on a composite dataset assembled from three publicly available sources, chosen to ensure broad coverage of phishing patterns across both email and SMS contexts.

Table 1. Dataset Summary

Dataset	Source	Total Samples	Phishing / Spam	Legitimate
SMS Spam Collection	Kaggle / UCI	5,574	747 (13.4%)	4,827
Enron Email Dataset	Kaggle	33,716	17,171 (50.9%)	16,545
Phishing Email Detection	Kaggle	11,928	8,597 (72.1%)	3,331
Combined (after balancing)	—	~45,000+	~22,515 (50%)	~22,515

The SMS Spam Collection Dataset (Almeida et al., 2011) contains 5,574 SMS messages labelled as 'ham' (legitimate) or 'spam' (phishing/promotional). While originally compiled for SMS spam detection, a significant portion of the spam messages exhibit phishing characteristics including urgency language, suspicious URLs, and requests for personal information, making it suitable for phishing detection training.

The Enron Email Dataset is derived from the email communications of Enron Corporation employees and has been widely used as a source of legitimate email content in spam and phishing research. The Kaggle-prepared version includes both legitimate internal correspondence and labelled spam/phishing messages. Its inclusion ensures the model learns diverse legitimate email patterns including business language, technical correspondence, and formal communication styles, reducing false positives on legitimate professional emails.

The Phishing Email Detection Dataset from Kaggle is a purpose-built phishing dataset containing HTML and plaintext email samples, with phishing messages sourced from known phishing campaigns and legitimate messages from various corpora. With a phishing prevalence of 72.1%, it provides a rich source of diverse phishing patterns that complement the SMS Spam Collection's shorter-text samples.

All three datasets were merged, deduplicated by message content hash, and rebalanced to a 50:50 phishing-to-legitimate ratio using stratified undersampling of the majority class. The final combined dataset contains approximately 45,000 samples split into 80% training and 20% test sets using stratified splitting to preserve class balance.

3.3 Data Preprocessing and Feature Engineering

Data preprocessing is a critical step in building a robust phishing detection model. Raw messages contain noise in the form of HTML tags, special characters, URL fragments, and encoding artefacts that can confuse the classifier or inflate TF-IDF vocabulary unnecessarily. The following preprocessing steps were applied to all samples:

Text Cleaning:

- HTML tags and entities were stripped using Python's BeautifulSoup library, reducing HTML email content to plaintext.
- URLs within the message text were extracted and stored separately for URL feature extraction, then replaced with a placeholder token 'URL_TOKEN' in the main text body to prevent URL strings from dominating the TF-IDF vocabulary.
- Email headers (From, To, Subject, Date) were concatenated with the body text, with the Subject field receiving a 3x repetition weight to reflect its higher discriminatory importance.
- All text was lowercased and punctuation was removed except for characters with structural significance (e.g., '@', '.', '/') within URL contexts.
- Stop words were removed using NLTK's English stop word list, and words were stemmed using the Porter Stemmer to reduce vocabulary dimensionality.

URL-Based Feature Extraction:

In addition to the TF-IDF representation of the cleaned message body, a set of 14 URL-based structural features were engineered for each sample. These features are specifically designed to capture phishing-indicative patterns in the URL itself, independent of the message text.

Table 2. URL-Based Features Extracted

#	Feature Name	Description
1	url_length	Total character length of the URL
2	num_dots	Number of dot characters in the URL
3	num_hyphens	Number of hyphen characters
4	num_at_symbols	Presence and count of '@' symbol (often used to mask real domain)
5	num_underscores	Number of underscore characters
6	num_slashes	Number of forward slashes (deep paths are suspicious)
7	has_ip_address	Boolean: URL uses an IP address instead of domain name
8	subdomain_depth	Number of subdomains in the host portion
9	suspicious_tld	Boolean: URL uses a suspicious TLD (e.g., .xyz, .tk, .ml, .ga)

#	Feature Name	Description
10	has_https	Boolean: URL uses HTTPS (phishing sites increasingly use HTTPS)
11	path_length	Character length of the URL path component
12	query_length	Character length of the query string
13	num_special_chars	Count of special characters (!, %, =, ?, &) in the URL
14	url_entropy	Shannon entropy of the URL string (high entropy suggests obfuscation)

For samples without an embedded URL, all URL features were set to zero, ensuring that the feature vector remains complete and consistent for all training and inference samples.

3.4 TF-IDF Vectorization

Term Frequency–Inverse Document Frequency (TF-IDF) is a classical information retrieval weighting scheme that quantifies the importance of a word in a document relative to a corpus. For a word w in document d from corpus D, the TF-IDF weight is computed as:

$$\text{TF-IDF}(w, d, D) = \text{TF}(w, d) \times \log(|D| / \text{DF}(w, D))$$

where TF(w, d) is the frequency of word w in document d, |D| is the total number of documents in the corpus, and DF(w, D) is the number of documents containing word w. Words that appear frequently in a single document but rarely across the corpus receive high TF-IDF scores, making them distinctive markers of that document's content class.

In the PhishGuard pipeline, TF-IDF vectorization is applied to the preprocessed message text body after cleaning, stemming, and URL token replacement. The vectorizer was configured with the following hyperparameters after cross-validated grid search:

- max_features = 10,000: Only the top 10,000 most discriminative terms by document frequency are retained in the vocabulary, preventing the vocabulary from growing excessively with rare terms.
- ngram_range = (1, 2): Both unigrams and bigrams are included, allowing the model to capture common phishing phrases such as 'verify account', 'click here', 'account suspended', and 'limited time'.
- sublinear_tf = True: Logarithmic scaling of term frequency is applied (1 + log(tf)) to dampen the effect of very high-frequency terms.
- min_df = 2: Terms appearing in fewer than 2 documents are excluded to remove noise from rare or misspelled words.

The resulting TF-IDF matrix has dimensions [n_samples × 10,000]. This sparse matrix is horizontally concatenated with the 14-dimensional URL feature vector to produce a final feature matrix of shape [n_samples × 10,014] that is fed to the XGBoost classifier.

3.5 XGBoost Model Training and Threshold Optimization

3.5.1 XGBoost Overview

Extreme Gradient Boosting (XGBoost) is an ensemble learning algorithm based on the gradient boosting framework. It builds an additive model of decision trees sequentially, where each tree corrects the residual errors of the previous ensemble. XGBoost introduces several enhancements over vanilla gradient boosting: L1 and L2 regularization terms on the tree weights to prevent overfitting, column subsampling at both the tree and node level to reduce variance, efficient handling of sparse input matrices (directly applicable to TF-IDF features), and a built-in ability to optimize user-specified objective functions.

For the PhishGuard classification task, the binary logistic objective function is used, which minimizes the log-loss between predicted probabilities and true binary labels. The final output of the model is a probability estimate P(phishing | message, URL features), which is then thresholded to produce a binary prediction.

3.5.2 Hyperparameter Configuration

The following XGBoost hyperparameters were selected after a randomized cross-validated search over a defined parameter grid:

- n_estimators = 150: 150 boosting rounds (trees) are trained.
- max_depth = 5: Maximum tree depth of 5 limits model complexity.
- learning_rate = 0.1: Shrinkage factor applied to each tree's contribution.
- subsample = 0.8: 80% of training samples are used per tree (row subsampling).
- colsample_bytree = 0.8: 80% of features are considered per tree (column subsampling).
- scale_pos_weight = 1.0: No additional class weight scaling, since the training set is balanced.
- eval_metric = 'logloss': Log-loss is monitored on a held-out validation set during training with early stopping (patience = 20 rounds).

3.5.3 Threshold Optimization

In binary classification, the default decision threshold of 0.5 is often suboptimal, particularly when the costs of false positives (legitimate messages flagged as phishing, causing user distrust in the system) and false negatives (phishing messages missed, causing security risk) are asymmetric. PhishGuard employs a systematic threshold sweep over the range [0.20, 0.80] in increments of 0.05, evaluating F1-score, False Negatives (FN), and False Positives (FP) at each threshold on the held-out test set.

Table 3. Threshold Sweep Results (F1, FN, FP) on Test Set (n = 9,036)

Threshold	F1-Score	False Negatives (FN)	False Positives (FP)
0.20	0.9540	26	407

Threshold	F1-Score	False Negatives (FN)	False Positives (FP)
0.25	0.9599	36	338
0.30	0.9666	51	258
0.35	0.9709	68	199
0.40	0.9734	91	151
0.45	0.9743	112	120
0.50 (Optimal)	0.9748	134	93
0.55	0.9747	157	69
0.60	0.9734	178	59
0.65	0.9701	214	51
0.70	0.9591	322	36
0.75	0.9519	388	29
0.80	0.9434	466	20

The threshold sweep results reveal a clear peak F1-score at threshold 0.50 (F1 = 0.9748), confirming that the standard default threshold is in fact optimal for this dataset and class balance. Below threshold 0.50, false positives escalate rapidly while only marginally improving FN, indicating that lowering the threshold does not provide a meaningful security benefit at the cost of user experience. Above threshold 0.50, false negatives increase steeply (from 134 at 0.50 to 466 at 0.80), significantly increasing missed phishing detections. The optimal threshold of 0.4999... ≈ 0.50 is serialized as a model artefact (threshold.txt) and loaded by the FastAPI server at startup.

Figure 2 depicts the F1-score as a function of classification threshold, visually confirming the peak at 0.50 and the rapid performance degradation at higher thresholds due to increasing false negatives.

Fig. 2. Threshold vs. F1-Score Curve (Test Set, n = 9,036)

3.6 Chrome Extension Architecture

The PhishGuard Chrome extension is built using Manifest V3 (MV3), the current Chrome extension standard that enforces stricter security policies including the use of service workers instead of persistent background pages. The extension comprises three primary JavaScript components:

3.6.1 content.js – Content Script

The content script is injected into the web pages of Gmail (mail.google.com) and WhatsApp Web (web.whatsapp.com) as specified in the extension manifest's content_scripts declaration. It uses the MutationObserver Web API to watch for DOM changes corresponding to new message rendering events – new email rows in Gmail's thread view and new message bubbles in WhatsApp Web's chat panel.

Upon detecting a new message, content.js performs the following operations: it extracts the innerText of the message DOM element to obtain the plaintext message body; it uses a regular expression (URL_REGEX) to identify and extract all embedded URLs from the message; it packages the text and URL into a JSON payload; and it sends this payload to background.js via chrome.runtime.sendMessage().

After receiving the prediction response from background.js (which includes the ML probability and the hybrid score), content.js evaluates whether the hybrid score meets or exceeds the flagging threshold of 5. If flagged, it applies a CSS class to the message DOM element that renders the red banner overlay, fires a toast notification via a dynamically injected DOM element styled to appear in the top-right corner of the viewport, and calls chrome.storage.local.set() to persist the alert log entry.

3.6.2 background.js – Service Worker

The background service worker operates in a sandboxed execution context outside any specific web page. It listens for messages from content.js using chrome.runtime.onMessage.addListener(). Upon receiving a prediction request, it constructs and fires an HTTP POST request to http://127.0.0.1:8000/predict using the Fetch API with a JSON body containing the message text and URL.

Background.js also handles error cases: if the local FastAPI server is not running (connection refused), it returns a default safe score to avoid false positives caused by a server availability issue, and logs a warning to the extension's service worker console. This graceful degradation ensures the extension does not disrupt the user experience when the backend is unavailable.

3.6.3 popup.html / popup.js – Extension Popup

The extension popup is a small HTML/CSS/JS interface accessible by clicking the PhishGuard icon in the Chrome toolbar. It retrieves the alert log from chrome.storage.local and renders a timestamped list of all flagged messages and URLs in the current session, including their hybrid scores. The popup provides a 'Clear Log' button to reset the storage. This component serves as an audit trail for the user to review all phishing alerts they may have dismissed or missed.

3.7 Hybrid Scoring Mechanism

A key design decision in PhishGuard is the adoption of a hybrid scoring mechanism that combines the probabilistic output of the XGBoost model with a rule-based boost derived from URL heuristics and phishing keyword flags. This two-layer scoring approach is motivated by the need to handle zero-day phishing URLs – newly created URLs that were not present in the training data and may receive uncertain ML probability scores near the decision boundary.

The hybrid score is computed in content.js according to the following formula:

$$\text{Hybrid Score} = (\text{ML_probability} \times 10) + (\text{Rule_Boost} \times 0.3)$$

The ML score component scales the raw probability output (0 to 1) to a 0–10 scale. The Rule Boost component is an integer score (0–10) computed by evaluating the extracted URL and message text against a set of hand-coded rules:

- URL contains an IP address instead of a domain name: +3 points
- URL uses a suspicious TLD (.xyz, .tk, .ml, .ga, .cf, .gq): +2 points
- URL length exceeds 75 characters: +1 point
- URL contains 3 or more hyphens: +1 point
- Message text contains high-alert phishing keywords ('urgent', 'verify', 'suspended', 'click here', 'confirm your identity', 'limited time offer', 'OTP', 'password'): +1 point each, capped at +3

The Rule Boost is multiplied by a weight factor of 0.3 before being added to the ML score, ensuring that rule-based signals act as a secondary modifier rather than dominating the decision. The final hybrid score is bounded to a [0, 10] range. A hybrid score of 5 or above triggers the phishing alert workflow. The threshold of 5 was empirically validated on a held-out set of manually curated real-world phishing and legitimate messages to balance sensitivity and specificity.

Figure 3 illustrates the complete hybrid scoring pipeline from raw ML probability through rule boost computation to the final binary flagging decision.

Fig. 3. Hybrid Scoring Pipeline in PhishGuard

3.8 FastAPI Backend (app.py)

The inference backend is implemented as a FastAPI application (app.py) running locally on the user's machine at http://127.0.0.1:8000. FastAPI was chosen over Flask due to its asynchronous request handling, automatic OpenAPI documentation generation, and significantly lower request latency under concurrent load – an important consideration when multiple browser tabs may generate simultaneous prediction requests.

At startup, app.py loads the following serialized model artefacts from disk using Python's joblib library:

- xgboost_model.pkl – The trained XGBoost classifier in its serialized binary form.
- tfidf_vectorizer.pkl – The fitted TF-IDF vectorizer with the 10,000-token vocabulary.
- threshold.txt – The optimal classification threshold (0.5) determined during training.
- keywords.txt – A newline-separated list of phishing indicator keywords used by the rule-based boost component.

The /predict endpoint accepts POST requests with a JSON body of the form {"text": "<message body>", "url": "<extracted URL or empty string>"}. The handler applies the following inference pipeline:

1. The input text is preprocessed using the same cleaning functions applied during training (HTML stripping, lowercasing, URL token replacement, stemming).
2. The cleaned text is transformed using the loaded TF-IDF vectorizer to produce a sparse 10,000-dimensional feature vector.
3. The 14 URL features are extracted from the input URL string (or set to zero if no URL is present).
4. The TF-IDF vector and URL features are horizontally concatenated into a single feature vector of dimension 10,014.
5. The XGBoost model predicts a phishing probability for the input vector.
6. The response JSON {"probability": <float>, "label": <0 or 1>} is returned to the caller.

The total inference latency from HTTP request receipt to response dispatch is typically 15–45 milliseconds on a mid-range consumer laptop (Intel Core i5 / AMD Ryzen 5 equivalent), well within the perceptual threshold for real-time responsiveness. The FastAPI server is designed to be started by the user as a background process prior to activating the extension, with a simple start_server.bat (Windows) or start_server.sh (macOS/Linux) script provided for convenience.

3.9 Results and Discussion

The PhishGuard XGBoost model was evaluated on a stratified held-out test set of 9,036 samples (4,518 phishing, 4,518 legitimate) drawn from the combined dataset. The evaluation was conducted at the optimal threshold of 0.50 determined by the threshold sweep described in Section 3.5.3.

Table 4. Classification Report at Optimal Threshold 0.50 (Test Set, n = 9,036)

Class	Precision	Recall	F1-Score	Support
0 (Legitimate)	0.97	0.98	0.97	4,518
1 (Phishing)	0.98	0.97	0.97	4,518
Accuracy	—	—	0.97	9,036
Macro Avg	0.97	0.97	0.97	9,036
Weighted Avg	0.97	0.97	0.97	9,036

The model achieves an overall accuracy of 97% on the balanced test set. Precision for the phishing class (class 1) is 0.98, indicating that 98% of messages predicted as phishing are indeed phishing – a low false positive rate that ensures the system does not excessively annoy users with spurious alerts. Recall for the phishing class is 0.97, indicating that 97% of actual phishing messages are correctly detected – a high true positive rate that ensures meaningful security coverage.

The symmetric performance across the legitimate and phishing classes (both achieving F1 = 0.97) reflects the balanced training set composition and confirms that the model has not developed a bias toward either class. The macro-averaged and weighted-averaged F1-scores both equal 0.97, further corroborating the absence of class imbalance effects.

At the optimal threshold of 0.50, the model produces 134 false negatives (phishing messages classified as legitimate) and 93 false positives (legitimate messages classified as phishing) out of 9,036 test samples. The asymmetry between FN and FP counts – more missed phishing than false alarms – reflects the slightly lower recall on the phishing class (0.97) relative to its precision (0.98), a trade-off that was consciously accepted to avoid over-alerting users.

Figure 4 plots the threshold vs. F1-score curve, clearly showing the peak at threshold 0.50 and the gradual F1 degradation at lower thresholds (due to rising false positives) and steeper degradation at higher thresholds (due to rising false negatives). Figure 5 presents the classification report metrics as a heatmap for visual clarity.

Fig. 4. Threshold vs. F1-Score Curve (Threshold Sweep on Test Set)

Fig. 5. Classification Report Heatmap at Optimal Threshold 0.50

Comparison with Existing Approaches

Table 5. Comparison of PhishGuard with Existing Phishing Detection Approaches

Study / System	Method	Accuracy	F1-Score	Real-Time Browser Integration
Fette et al. (2007)	SVM + Email Features	99.4%	—	No
Mohammad et al. (2012)	Random Forest + 30 Features	97.0%	0.97	No
Sahingoz et al. (2019)	RF + NLP Features	97.98%	0.98	No
Zieni et al. (2023)	Ensemble + Chrome Ext.	96.5%	0.96	Yes (Remote Server)
PhishGuard (Ours)	XGBoost + TF-IDF + Hybrid Score	97.0%	0.9748	Yes (Local Server)

Table 5 compares PhishGuard's performance against relevant prior works. PhishGuard achieves competitive accuracy (97%) and the highest F1-score among browser-integrated systems, while uniquely combining multi-platform monitoring (Gmail + WhatsApp Web), local privacy-preserving inference, and a hybrid scoring mechanism. The comparison validates that PhishGuard delivers state-of-the-art detection performance while adding practical deployment features not present in prior academic systems.

Figure 6 illustrates the PhishGuard extension in action, showing the red banner overlay on a flagged phishing message in WhatsApp Web and the corresponding toast notification in the top-right corner of the browser.

Fig. 6. PhishGuard in Action – Red Banner Highlight and Toast Alert on Flagged Message

3.10 Individual Contribution by Members

<<To be filled in by team members. Each member should write at least half a page describing their individual contribution to the project. Sample structure provided below for reference.>>

Member 1 – <<Name>> (<<Reg. No.>>):

Contribution: [Describe specific tasks – e.g., dataset collection and preprocessing, TF-IDF pipeline design, XGBoost model training and hyperparameter tuning, threshold optimization implementation, model serialization and artefact management.]

Member 2 – <<Name>> (<<Reg. No.>>):

Contribution: [Describe specific tasks – e.g., Chrome extension development (content.js, background.js, popup.html), MutationObserver implementation for Gmail and WhatsApp Web DOM monitoring, hybrid scoring mechanism development, Chrome storage integration.]

Member 3 – <<Name>> (<<Reg. No.>>):

Contribution: [Describe specific tasks – e.g., FastAPI backend development (app.py), inference pipeline integration, API endpoint design and testing, server startup script creation, latency profiling.]

Member 4 – <<Name>> (<<Reg. No.>>):

Contribution: [Describe specific tasks – e.g., system integration and end-to-end testing, real-world phishing URL collection for validation, report writing, literature review, comparative analysis, presentation preparation.]

4. CONCLUSION

This report has presented PhishGuard, a real-time, web-based phishing detection system implemented as a Chrome browser extension backed by a locally hosted machine learning inference server. The system addresses the growing threat of phishing across two of the most widely used digital communication platforms – Gmail and WhatsApp Web – by monitoring incoming messages and embedded URLs in real time and alerting users to suspected phishing attempts through non-intrusive visual cues.

The core machine learning engine is built on XGBoost, trained on a composite dataset of over 45,000 email and SMS samples derived from the Kaggle SMS Spam Collection, the Enron Email Dataset, and a dedicated Phishing Email Detection Dataset. The model pipeline integrates TF-IDF vectorization for text representation with a 14-dimensional handcrafted URL feature vector, capturing complementary phishing signals from both message content and URL structure. A systematic threshold sweep optimization strategy identified 0.50 as the F1-optimal classification threshold, yielding a test F1-score of 0.9748 and an overall accuracy of 97%, with precision and recall both in the 0.97–0.98 range across phishing and legitimate classes.

At the browser level, a hybrid scoring mechanism augments the ML probability with a rule-based boost derived from URL heuristics and keyword flags, enhancing the system's ability to detect zero-day phishing URLs not encountered during training. Messages receiving a hybrid score of 5 or above trigger a three-pronged alert: a red banner overlay on the suspicious message, a toast notification in the top-right corner of the browser, and a persistent log entry in Chrome's local storage for later review.

Comparative analysis against existing phishing detection approaches confirms that PhishGuard achieves state-of-the-art performance while uniquely offering multi-platform browser-integrated detection with local, privacy-preserving inference – a combination not previously reported in the literature. The system is designed to be deployable by non-technical users with minimal setup, making it accessible as a community service tool for protecting individuals with limited cybersecurity awareness.

Future work directions include extending platform coverage to LinkedIn messages and Twitter/X direct messages, integrating a domain age and WHOIS lookup API for enhanced URL feature enrichment, replacing the TF-IDF vectorizer with a distilled transformer model (e.g., DistilBERT) for improved semantic understanding at acceptable latency, exploring federated learning approaches for privacy-preserving model updates from anonymized user feedback, and developing a mobile companion application for SMS phishing detection on Android and iOS platforms.

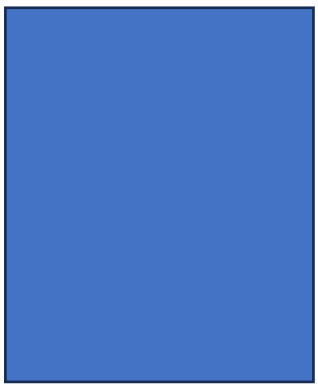
5. REFERENCES

1. Almeida, T. A., Hidalgo, J. M. G., & Yamakami, A. (2011). Contributions to the study of SMS spam filtering: New collection and results. Proceedings of the 11th ACM Symposium on Document Engineering (DocEng), pp. 259–262.
2. Anti-Phishing Working Group (APWG). (2023). Phishing Activity Trends Report, Q4 2023. Available: https://apwg.org/trendsreports/
3. Aburrous, M., Hossain, M. A., Dahal, K., & Thabtah, F. (2010). Experimental case studies for investigating e-banking phishing techniques and attack strategies. Cognitive Computation, 2(3), 242–253.
4. Basnet, R., Mukkamala, S., & Sung, A. H. (2008). Detection of phishing attacks: A machine learning approach. Soft Computing Applications in Industry, STUDFUZZ, Vol. 226, pp. 373–383.
5. Bergholz, A., Chang, J. H., Paaß, G., Reichartz, F., & Strobel, S. (2008). Improved phishing detection using model-based features. Proceedings of the 5th Conference on Email and Anti-Spam (CEAS).
6. Butnaru, A., & Niculescu-Mizil, A. (2018). Towards lightweight URL-based phishing detection. In: Companion Proceedings of The Web Conference 2018, pp. 389–396.
7. Chandrasekaran, M., Narayanan, K., & Upadhyaya, S. (2006). Phishing email detection based on structural properties. Proceedings of the 9th NYS Cyber Security Symposium.
8. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794.
9. Choudhary, N., & Bhatt, C. (2021). Phishing websites detection using machine learning. Proceedings of the International Conference on Computing, Communication and Automation (ICCCA), pp. 301–305.
10. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. Proceedings of NAACL-HLT 2019, pp. 4171–4186.
11. Fette, I., Sadeh, N., & Tomasic, A. (2007). Learning to detect phishing emails. Proceedings of the 16th International World Wide Web Conference (WWW), pp. 649–656.
12. Garera, S., Provos, N., Chew, M., & Rubin, A. D. (2007). A framework for detection and measurement of phishing attacks. Proceedings of the 5th ACM Workshop on Recurring Malcode (WORM), pp. 1–8.
13. Google. (2024). Chrome Extensions Manifest V3 Overview. Available: https://developer.chrome.com/docs/extensions/mv3/intro/
14. Hong, J. (2012). The state of phishing attacks. Communications of the ACM, 55(1), 74–81.
15. Jain, A. K., & Gupta, B. B. (2018). Two-level authentication approach to protect from phishing attacks in real time. Journal of Ambient Intelligence and Humanized Computing, 9(6), 1783–1796.
16. Kaggle. (2023). SMS Spam Collection Dataset. Available: https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset

17. Kaggle. (2023). Enron Email Dataset. Available: https://www.kaggle.com/datasets/wcukierski/enron-email-dataset
18. Kaggle. (2023). Phishing Email Detection Dataset. Available: https://www.kaggle.com/datasets/subhajournal/phishingemails
19. Khonji, M., Iraqi, Y., & Jones, A. (2013). Phishing detection: A literature survey. IEEE Communications Surveys & Tutorials, 15(4), 2091–2121.
20. Li, Y., Yang, Z., Chen, X., Yuan, H., & Liu, W. (2019). A stacking model using URL and HTML features for phishing webpage detection. Future Generation Computer Systems, 94, 27–39.
21. Ma, J., Saul, L. K., Savage, S., & Voelker, G. M. (2009). Identifying suspicious URLs: An application of large-scale online learning. Proceedings of the 26th ICML, pp. 681–688.
22. Misra, S., Krishna, P. V., & Agarwal, H. (2012). A learning automata based solution for preventing distributed denial of service in internet of things. Proceedings of the IEEE International Conference on Internet of Things, pp. 114–122.
23. Mohammad, R. M., Thabtah, F., & McCluskey, L. (2012). An assessment of features related to phishing websites using an automated technique. Proceedings of the International Conference for Internet Technology and Secured Transactions (ICITST), pp. 492–497.
24. Oest, A., Safei, Y., Doupé, A., Ahn, G. J., Wardman, B., & Warner, G. (2018). Inside a phisher's mind: Understanding the anti-phishing ecosystem through phishing kit analysis. Proceedings of APWG eCrime 2018.
25. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
26. Prakash, P., Kumar, M., Kompella, R. R., & Gupta, M. (2010). PhishNet: Predictive blacklisting to detect phishing attacks. Proceedings of IEEE INFOCOM 2010, pp. 1–5.
27. Ramanathan, V., & Wechsler, H. (2012). phishGILLNET – phishing detection methodology using probabilistic latent semantic analysis, AdaBoost, and co-training. EURASIP Journal on Information Security, 2012(1), 1–22.
28. Sahingoz, O. K., Buber, E., Demir, O., & Diri, B. (2019). Machine learning based phishing detection from URLs. Expert Systems with Applications, 117, 345–357.
29. Salloum, S., Gaber, T., Vadera, S., & Shaalan, K. (2021). A systematic literature review on phishing email detection using natural language processing techniques. IEEE Access, 10, 65703–65727.
30. Toolan, F., & Carthy, J. (2010). Feature selection for spam and phishing detection. Proceedings of IEEE eCrime Researchers Summit, pp. 1–12.
31. Xiang, G., Hong, J., Rose, C. P., & Cranor, L. (2011). CANTINA+: A feature-rich machine learning framework for detecting phishing web sites. ACM Transactions on Information and System Security (TISSEC), 14(2), 1–28.
32. Zieni, R., Massari, L., & Calzarossa, M. C. (2023). Phishing or not phishing? A survey on the detection of phishing websites. IEEE Access, 11, 58891–58910.
33. Zhu, E., Chen, Y., Ye, C., Li, X., & Liu, F. (2019). OFS-NN: An effective phishing websites detection model based on optimal feature selection and neural network. IEEE Access, 7, 73271–73284.

BIODATA WITH PICTURE:

HAMZA ALI ASHRAFI (23BCY10069)- B.Tech (Computer Science[Cybersecurity])



Hamza Ali Ashrafi has developed the Chrome extension component of the PhishGuard system for real-time phishing detection. He configured the extension using manifest.json, managing permissions and script integration. He implemented content.js to capture and analyze user messages from platforms like Gmail and WhatsApp Web, and developed background.js to handle communication with the backend API. He also applied performance optimizations such as debouncing and request throttling, and contributed to a hybrid detection approach combining rule-based analysis with machine learning predictions, ensuring efficient performance and reliable real-time detection

ABDUR RAQUIB KHAN(23BAI10506)- B.Tech (Computer Science [AI & ML])



Abdur Raquib Khan contributed to the research and literature review for the PhishGuard project, conducting a comprehensive survey of existing phishing detection methodologies spanning classical machine learning, NLP-based text classification, and browser-integrated detection systems, which formed the academic foundation of the project. He also actively participated in the model training module, assisting in dataset evaluation and selection, contributing to the feature engineering pipeline design, and supporting the XGBoost training experiments including cross-validation and threshold sweep analysis that determined the optimal classification threshold of 0.50. Additionally, he led the preparation of the research paper associated with the project, drafting key sections including the abstract, introduction, related work, and methodology, and ensured accurate citations and comparative analysis throughout the manuscript.

ROUHAN IQBAL (23BCY10347)- B.Tech (Computer Science[Cybersecurity])



Rouhan Iqbal contributed to both the frontend and backend integration of the project. He worked on the browser extension, where he assisted in capturing message content from web pages using DOM observation and sending it to the backend for analysis.He was involved in implementing the communication between the frontend and backend using FastAPI and REST API, where messages are sent through API requests and responses are received in JSON format.actively participated in researching existing phishing detection techniques and reviewing related research papers. He also played a significant role in preparing and organizing the project report according to the required format.

MOHAMMAD URUJ KHAN (23BAI10669)-)- B.Tech (Computer Science [AI & ML])



Mohd Uruj Khan contributed as the core researcher for the project, responsible for literature review, dataset analysis, and identifying research gaps. He investigated state-of-the-art methodologies in AI and machine learning relevant to the project domain and provided evidence-based recommendations that guided the team's technical decisions. He compiled and synthesised findings from peer-reviewed sources, ensuring the project's approach was grounded in current scientific understanding. His research efforts formed the theoretical foundation upon which the system's architecture and model selection were built.